A PROJECT REPORT
on

QLens: From a Messy Spreadsheet to a Verified Third-Normal-Form Database

Submitted to Manipal University Jaipur
Towards the partial fulfillment for the Award of the Degree of

B. Tech Computer Science and Engineering

2026–2027

By

Aditya Gaur
Reg. No. 2502051500

Aditya Choudhary
Reg. No. 2502051502

Semester III, Section K3 · Relational Database Management Systems

[University logo: figures/muj-logo.png, 4.56 in × 0.9 in]

Under the guidance of

Ms. Surabhi Agrawal

Department of Artificial Intelligence and Machine Learning
Manipal University Jaipur

Jaipur, Rajasthan

Abstract

Small offices keep their records in a single, ever-growing spreadsheet. A coaching institute's fee register repeats every teacher's phone number on hundreds of rows, spells the same student three different ways, and needs an hour of filtering to answer "who has not paid for October?". These are the update, insertion and deletion anomalies that normalization theory was invented to remove, but the people who suffer from them have never heard the terms and the tools they use do not help: database front-ends expect the user to design the tables, and chat-style assistants answer one question without leaving a correct database behind.

QLens is a local web application that takes such a sheet and produces a normalized PostgreSQL database from the data itself. The file is copied unchanged into a raw table and a typed staging table, with column types inferred by SQL. Candidate keys and functional dependencies are then discovered inside the database by PL/pgSQL functions (`qlens.check_fd`, `qlens.is_unique`) that group the staged rows with dynamic SQL; a dependency that holds on all but a few rows is reported as a data inconsistency together with the rows that break it. Near-duplicate spellings are found with `pg_trgm` trigram similarity over a GIN index, and every repair is logged. From the accepted exact dependencies QLens computes a minimal cover and runs Bernstein's 3NF synthesis, extended with a key relation so that the decomposition is always lossless, and turns the result into tables with surrogate keys, `UNIQUE NULLS NOT DISTINCT` natural keys and foreign keys. The design is applied in a single transaction and accepted only if a view that joins the new tables back together returns exactly the staged rows, compared as multisets with `EXCEPT ALL` in both directions.

QLens keeps its own metadata in a normalized `qlens` schema of 22 tables with CHECK constraints, generated columns, triggers and views that use window functions. Each workspace has its own schemas and its own low-privilege PostgreSQL login role, so user queries are isolated by privileges rather than by convention. On the bundled 3,000-row fee register the analysis finds the two composite candidate keys, six core dependencies and the 13 planted faulty rows, and proposes the textbook four-table design (`students`, `batches`, `teachers`, `fee_register`); applying it rebuilds all 3,000 rows exactly. The algorithms are verified by unit tests on textbook examples, by 900 randomized synthesis cases checked against a chase-based lossless-join oracle and a brute-force 3NF test, and by end-to-end tests that apply the generated SQL to randomly generated sheets (302 backend and 63 frontend tests pass). Applied designs are queried in a sandboxed, read-only SQL editor; a tool-using assistant is in progress, and every capability is marked with its build status.

Keywords: functional dependency discovery, third normal form, Bernstein synthesis, lossless-join decomposition, PostgreSQL, PL/pgSQL, role-based isolation, data cleaning, `pg_trgm`.

Table of Contents

Abstract i

Table of Contents ii

List of Figures iii

List of Tables iv

1 Introduction 1

1.1 Scope of the Work 2

1.2 Product Scenarios 2

2 Requirement Analysis 4

2.1 Functional Requirements 4

2.2 Non-functional Requirements 4

2.3 Use Case Scenarios 5

2.4 Other Software Engineering Methodologies 5

3 System Design 7

3.1 Design Goals 7

3.2 System Architecture 7

3.3 Detailed Design Methodologies 7

4 Conclusion 21

References 22

A Sample Datasets and Screens 23

List of Figures

Figure 1.1 Landing page 2

Figure 2.1 Use-case diagram 6

Figure 3.1 Layered architecture 8
Figure 3.2 ER diagram of the metadata schema 9
Figure 3.3 Relational schema of `qlens` 10
Figure 3.4 Normalization pipeline 15
Figure 3.5 Fee-register dependency diagram 16
Figure 3.6 ER diagram of the generated fee-register database 17
Figure 3.7 Query sequence 19
Figure 3.8 Design canvas 20

Figure A.1 Screen gallery 23

List of Tables

Table 2.1 Functional requirements 4
Table 2.2 Non-functional requirements 5
Table 2.3 Test summary 6

Table 3.1 Keys and relationships 11
Table 3.2 Integrity rules 12
Table 3.3 Secondary indexes 12
Table 3.4 SQL used by each feature 14

Table A.1 Sample datasets 23

CHAPTER 1

Introduction

Small offices run on one ever-growing spreadsheet. A coaching institute's fee register is typical: one row per student per month, with the student's name and phone, the batch, the batch teacher and the teacher's phone, the month, the fee paid and the payment mode. Over a year it reaches thousands of rows and develops the classic anomalies of an unnormalized relation [1, 2]:

- **Update anomaly.** A teacher's phone number is repeated on every row of every student of that teacher; when it changes, a few old rows are always missed, and the sheet now holds two numbers.
- **Inconsistency.** "Riya Sharma" and "riya sharma" are counted as two students, so totals and dues are wrong.
- **Insertion and deletion anomalies.** A new teacher cannot be recorded until a student pays; deleting the last row of a batch deletes the only record of who teaches it.
- **Query cost.** "Who has not paid for October?" takes an hour of filtering.

In database terms the sheet violates third normal form: `teacher_phone` depends on `teacher`, which depends on `batch`, which depends on the student, which is only part of the key (`student_name`, `month`). The people who suffer from this do not know the vocabulary, and the tools they use do not help: database front-ends such as Airtable or NocoDB expect the user to design the tables, and chat assistants answer one question about a file without leaving a correct database behind.

QLens closes this gap. It reads the sheet into PostgreSQL, discovers from the data itself which columns determine which (functional dependencies) and which columns identify a row (candidate keys), shows the rows that break an almost-true rule as inconsistencies, finds near-duplicate spellings, and then synthesizes a third-normal-form design [3]. With one click the design is created as real tables with primary, unique and foreign keys, the data is migrated in a single transaction, and the migration is accepted only if joining the new tables back gives exactly the original rows. The objectives were therefore:

O1. Do the analysis *inside* the database, with SQL and PL/pgSQL, so that its correctness rests on the same engine that will hold the data.

O2. Produce designs that are provably in 3NF and lossless, and verify losslessness on the real rows.

O3. Isolate every user and workspace by PostgreSQL privileges, not by application convention.

O4. Present every result in plain language, for users who have never written SQL.

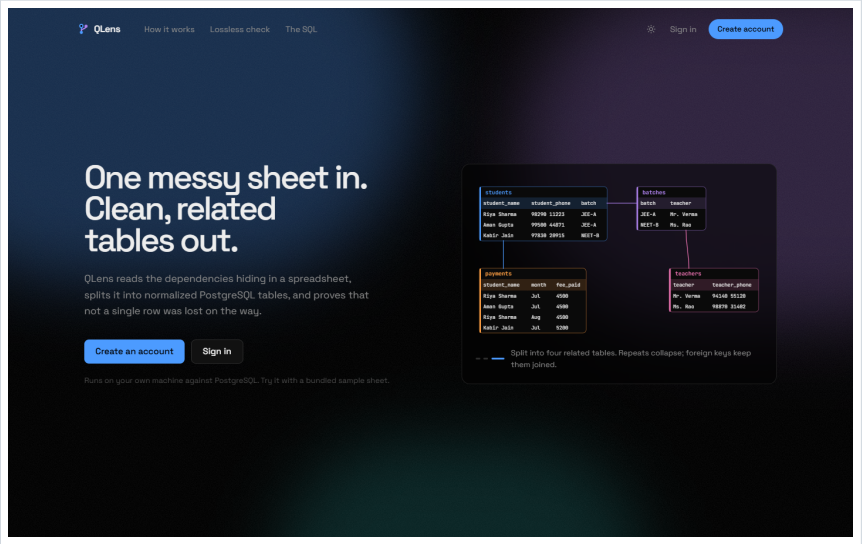


Figure 1.1: The QLens landing page: one messy sheet in, clean related tables out.

1.1 Scope of the Work

In scope. Import of one CSV or XLSX sheet per workspace (up to 20 MB, 100,000 rows and 60 columns); SQL type inference and profiling; discovery of candidate keys of up to three columns and of exact and approximate functional dependencies with a left-hand side of up to two columns; near-duplicate detection and repair with an audit log; minimal cover and 3NF synthesis; generation of the DDL; a transactional apply with a lossless check; a sandboxed, read-only query editor with history; and three bundled sample sheets with planted errors (a coaching fee register, a kirana shop's sales and a hostel mess register) so that the product can be demonstrated without private data. QLens keeps its own records in a normalized metadata schema, which is itself a main subject of this report.

Out of scope. Other database engines, multi-sheet workbooks, live collaboration, writes from the query editor, normal forms beyond 3NF (BCNF can lose dependencies; 3NF synthesis cannot), and the hardening needed for public multi-tenant hosting. QLens runs locally on one laptop against a private PostgreSQL 18 cluster.

Status. The report describes the system at the time of submission and marks each capability as Built (implemented and covered by automated tests: 302 backend tests and 63 frontend tests pass), In progress (partly implemented) or Planned (designed, not yet started). Nothing is described as working unless it is marked Built.

1.2 Product Scenarios

The coaching-institute owner. Built The owner imports the fee register (3,000 rows: 300 students, 8 batches, 6 teachers, 10 months). QLens reports the keys (`student_name`, `month`) and (`student_phone`, `month`), six core dependencies, and two inconsistencies: four students spelled differently on 8 rows, and Mr. Gupta's old phone number on 5 rows. After "accept

all suggestions" it proposes `students`, `batches`, `teachers` and `fee_register`; applying the design rebuilds all 3,000 rows exactly. A teacher's phone is now stored once.

The developer migrating legacy data. Built A developer uploads an undocumented CSV export and reads off the keys and dependencies that actually hold, with the share of violating rows next to each approximate one, which shows where the legacy data cannot be trusted. The proposal comes with generated `CREATE TABLE` statements that can be copied into the application's migrations, or QLens can migrate the data itself and prove that nothing was lost.

The student learning normalization. Built A student opens the hostel mess sample and sees why `room` determines `hostel_block`, how the minimal cover is derived and why each table of the proposal exists, then queries the result with joins, grouping and window functions. A clause-by-clause visualizer and a performance laboratory with real execution plans are Planned.

CHAPTER 2

Requirement Analysis

The requirements come from the anomalies of Chapter 1, from the three target users and from the three sample sheets, whose planted errors double as acceptance tests.

2.1 Functional Requirements

Table 2.1: Functional requirements and their implementation status.

ID	Requirement	Status
FR-1	**Accounts and workspaces.** Sign-up and sign-in; each workspace gets its own schemas and database role.	Built
FR-2	**Import.** CSV (encoding and delimiter detected) or XLSX; raw cells kept unchanged in `_raw`.	Built
FR-3	**Type inference and profile.** Infer integer, numeric, boolean, date (incl. day-first) or text in SQL; null and distinct counts.	Built
FR-4	**Key and FD discovery.** Candidate keys (1–3 columns), exact and approximate dependencies (LHS 1–2), computed in PostgreSQL.	Built
FR-5	**Clean-up.** Near-duplicate groups via trigram similarity; conflicting rows of approximate dependencies with a suggested value; every edit logged.	Built
FR-6	**3NF proposal.** Minimal cover, Bernstein synthesis, surrogate and foreign keys, names, explanations, DDL; renames while a draft.	Built
FR-7	**Apply.** Create and load the tables in one transaction; commit only if the join of the new tables equals the cleaned rows.	Built
FR-8	**Design canvas.** Draggable ER diagram with rename, SQL panel and apply.	Built
FR-9	**Sandboxed queries and history.** Read-only SELECTs as the workspace role, timeout and row cap; saved queries and run log.	Built
FR-10	**Assistant.** Optional Gemini command bar: answers questions, edits the draft only through validated tools.	In progress
FR-11	**Performance lab and step-through.** Plan tree from `EXPLAIN ANALYZE`, index experiments, clause-by-clause row counts.	Planned
FR-12	**Samples.** Three generated sheets with known, planted problems.	Built

2.2 Non-functional Requirements

A text filter such as “the statement must start with `SELECT`” is easily bypassed with a comment, a CTE or a function that writes. NFR-1 is therefore met by layers that fail independently: privileges, a read-only transaction, a timeout and a row cap, all enforced by PostgreSQL itself [4].

Table 2.2: Non-functional requirements and the mechanism that meets each.

ID	Quality	Mechanism
NFR-1	Isolation	One PostgreSQL `LOGIN` role per workspace with privileges only on its schemas; read-only sessions, 5 s `statement_timeout`, 1,000-row cap.
NFR-2	Correctness	3NF and losslessness by construction, property-tested; `EXCEPT ALL` check on the real rows at apply time.
NFR-3	Integrity	The database rejects bad metadata: `CHECK`, `UNIQUE`, foreign keys, generated columns, partial unique indexes.
NFR-4	Atomicity	Import, every clean-up and apply each run in one transaction; failure leaves no partial state.
NFR-5	Performance	Analysis of the 3,000-row sample runs inside PostgreSQL within the 5 s bound asserted by a test; trigram search uses a GIN index.
NFR-6	Auditability	`_raw` is never edited; each fix is logged in `dataset_fixes` with old value, new value and row count.
NFR-7	Security	Argon2id password hashes; only SHA-256 digests of session tokens stored; every identifier quoted, every value bound.
NFR-8	Usability	Plain-English findings, light and pure-black themes, motion that respects reduced-motion settings.

2.3 Use Case Scenarios

Figure 2.1 shows the use cases. Importing, reviewing data health, cleaning, generating and applying a design, and querying the result are Built; asking the assistant is In progress; studying a query in the performance lab or step-through is Planned. Apply is the critical one: it either rebuilds every row exactly or changes nothing (Section 3.3.10).

2.4 Other Software Engineering Methodologies

Iterative, test-driven delivery. The system was built in thin vertical increments in the order of value: database foundation and accounts, then import, discovery, clean-up, synthesis, proposal and apply, then querying. Each increment started from failing tests [5] and was followed by a review whose findings became regression tests; for example, a randomized test showed that plain synthesis without a key relation was lossy in 74 of 3,000 random cases, which led to the key step of Algorithm 3.1. The schema is managed by numbered SQL migrations applied by a small runner, and the test database is rebuilt from them at the start of every session.

Test results. Table 2.3 summarizes the suites. Database tests run against a real PostgreSQL 18 instance, because the central logic lives in SQL and cannot be meaningfully mocked.



Figure 2.1: Use-case diagram. The three target groups specialise a signed-in user and inherit the common flow (import, analyse, clean, design, apply, query); each group adds the use cases it motivates. Colours follow the pipeline phases of Figure 3.4.

Table 2.3: Automated tests (latest full run: 302 backend cases and 63 frontend tests, all passing).

Area	Cases	What is verified
Schema, migrations, auth, workspaces	39	constraints, generated columns, triggers, views; role and schema set-up
Import (parse, stage, routes, samples)	39	encodings, delimiters, header names, type inference, planted errors
Discovery and clean-up	53	`check_fd`, keys, noise rules, trigram groups, fixes, rollback
FD algebra	19	closure, minimal cover, synthesis; 900 random cases checked for 3NF and losslessness
Naming, proposal and apply	79	FK substitution, renames, generated SQL on random sheets, apply, rollback, catalog
Query runner	73	parser checks and denylist, sandbox isolation, time-out, row cap, value serialization, run recording
Frontend (Vitest)	63	API client, formatting, lane colours, dashboard and analysis view models

CHAPTER 3

System Design

3.1 Design Goals

G1. **Correct by construction, verified on data.** Designs are 3NF and lossless by the algorithm, and every apply re-checks losslessness on the real rows before committing.

G2. **SQL in plain sight.** No ORM: constraints, PL/pgSQL, triggers, views, dynamic SQL and catalog queries are hand-written, and the generated SQL is shown to the user.

G3. **Work where the data is.** Profiling, dependency checks and duplicate search run inside PostgreSQL; only the set algebra of synthesis runs in Python.

G4. **Isolation by privilege.** A user's query reaches only its own workspace because the database role lacks the privileges, not because the application filters text.

G5. **Non-destructive and auditable.** The raw import is never modified; every repair is a logged, transactional edit of a separate staging copy.

3.2 System Architecture

QLens is a three-tier web application whose bottom tier does unusually much work (Figure 3.1). The *client* is a React 19 and TypeScript single-page application [6] with React Flow diagrams and a CodeMirror SQL editor; it talks JSON to `/api/*` with the session in an `httpOnly` cookie. The *application tier* is FastAPI on Python 3.13 [7] with psycopg 3 [8] and pglast [9]; each request runs in one transaction on a pooled connection as the owning role `qlens_app`, committed before the response is sent. The *data tier* is a private PostgreSQL 18 cluster [4] holding the `qlens` metadata schema and, per workspace, the schemas `ws_<id>_src` (`_raw`, never edited, and `_staging`, typed and cleaned), `ws_<id>` (the generated tables) and `ws_<id>_lab` (Planned), plus one login role `qlens_ws_<id>` (Section 3.3.6).

3.3 Detailed Design Methodologies

The design has two halves: the *database design* of QLens itself and of the databases it generates (Section 3.3.1 to Section 3.3.7), and the *methods* that turn a sheet into such a database (Section 3.3.8 onwards).

3.3.1 Conceptual Design: ER Diagram

QLens records everything it knows in the `qlens` schema: 22 tables plus a migrations log (Figure 3.2). Users own workspaces; a workspace has at most one dataset; a dataset has columns, candidate keys, functional dependencies, duplicate groups and a log of fixes; designs are proposals with their tables, columns and unique keys; and the query side records saved queries, runs, plans and index experiments. Multi-column keys and dependency left-hand sides are many-to-many relationships between a key (or dependency) and dataset columns,

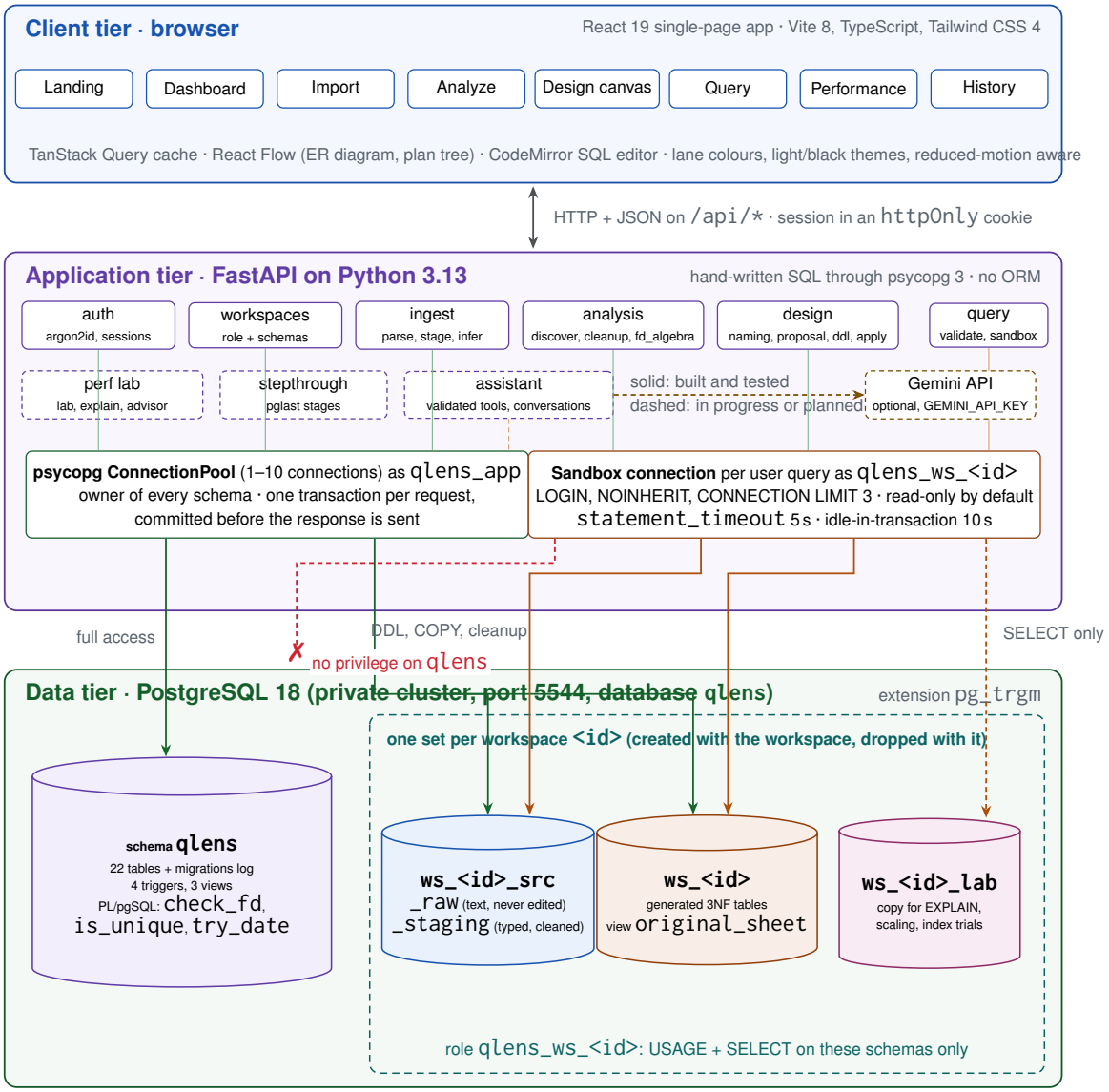


Figure 3.1: Layered architecture of QLens. Application code reaches PostgreSQL through two kinds of connection: the shared pool as the owning role, and a per-query sandbox connection as the workspace's own low-privilege LOGIN role.

resolved by junction tables.

3.3.2 Relational Schema

Figure 3.3 gives the logical schema in textbook notation. Every surrogate key is `bigint GENERATED ALWAYS AS IDENTITY`; every timestamp is `timestamptz NOT NULL DEFAULT now()`.

3.3.3 Keys and Relationships

Besides the primary keys, the alternate keys are `users(email)`, `workspaces(owner_id, name)`, `datasets(workspace_id)`, `dataset_columns(dataset_id, name)` and `(dataset_id, ordinal)`, `duplicate_groups(dataset_id, column_id, canonical_value)`, `proposed_tables(proposal_id, name)`, `proposed_columns(table_id, name)`, `saved_queries(workspace_id, title)` and `nl_questions(query_run_id)`. The 31 foreign keys form the relationships of Table 3.1.

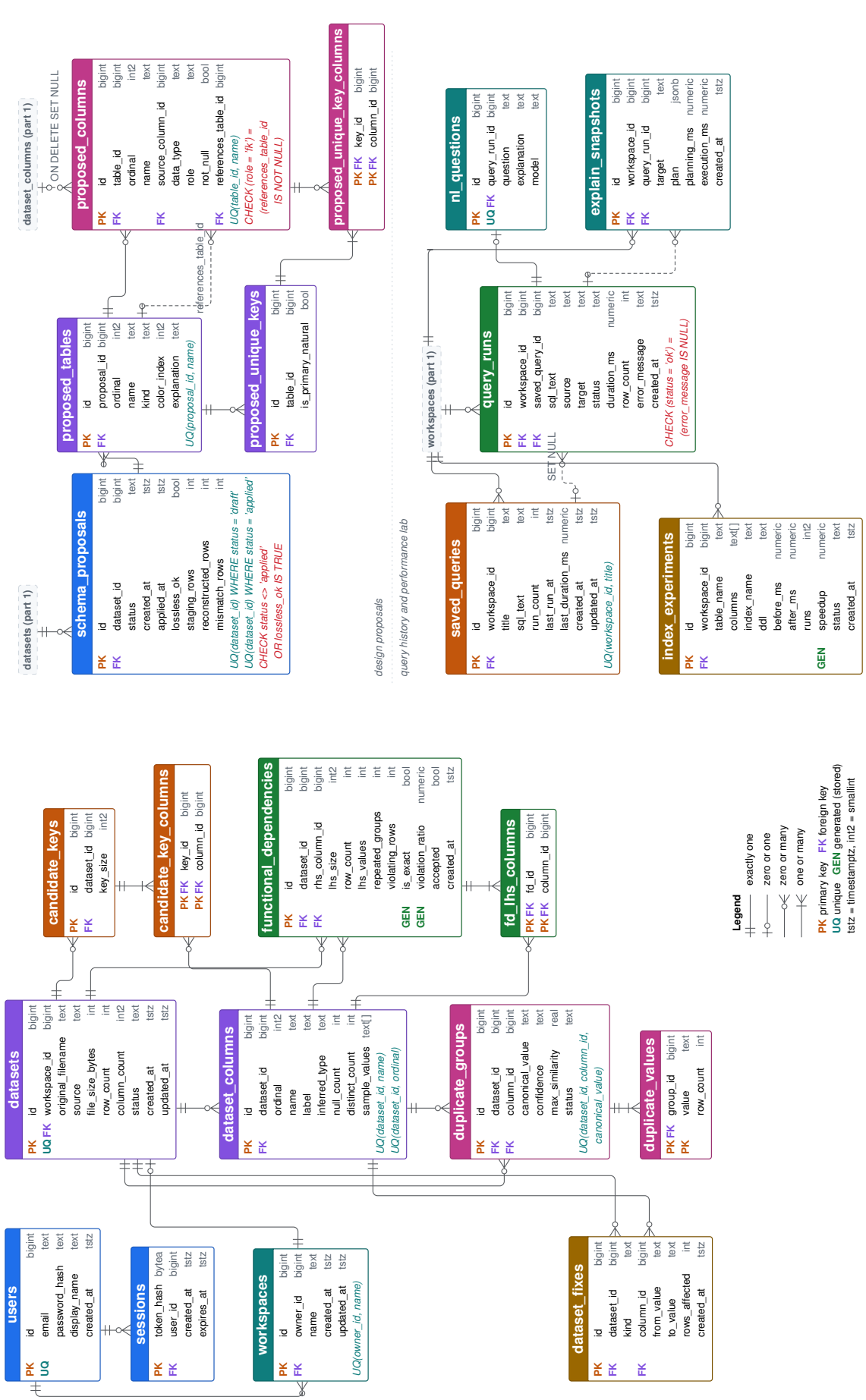


Figure 3.2: Entity–relationship diagram of the `qlens` metadata schema (22 tables) in crow's-foot notation. Left: accounts, workspaces, the dataset and the analysis results. Right: proposed designs, query history and the performance lab; grey boxes repeat tables from the left, dashed relationships are optional (nullable foreign keys).

Accounts and workspaces

users(id, email, password_hash, display_name, created_at)

sessions(token_hash, *user_id*†, created_at, expires_at)

workspaces(id, *owner_id*†, name, created_at, updated_at)

Dataset and analysis

datasets(id, *workspace_id*†, original_filename, source, file_size_bytes, row_count, column_count, status, created_at, updated_at)

dataset_columns(id, *dataset_id*†, ordinal, name, label, inferred_type, null_count, distinct_count, sample_values)

candidate_keys(id, *dataset_id*†, key_size)

candidate_key_columns(*key_id*†, *column_id*†)

functional_dependencies(id, *dataset_id*†, *rhs_column_id*†, lhs_size, row_count, lhs_values, repeated_groups, violating_rows, is_exact, violation_ratio, accepted, created_at)

fd_lhs_columns(*fd_id*†, *column_id*†)

duplicate_groups(id, *dataset_id*†, *column_id*†, canonical_value, confidence, max_similarity, status)

duplicate_values(*group_id*†, value, row_count)

dataset_fixes(id, *dataset_id*†, kind, *column_id*†, from_value, to_value, rows_affected, created_at)

Proposed designs

schema_proposals(id, *dataset_id*†, status, created_at, applied_at, lossless_ok, staging_rows, reconstructed_rows, mismatch_rows)

proposed_tables(id, *proposal_id*†, ordinal, name, kind, color_index, explanation)

proposed_columns(id, *table_id*†, ordinal, name, *source_column_id*†, data_type, role, not_null, *references_table_id*†)

proposed_unique_keys(id, *table_id*†, is_primary_natural)

proposed_unique_key_columns(*key_id*†, *column_id*†)

Queries and performance

saved_queries(id, *workspace_id*†, title, sql_text, run_count, last_run_at, last_duration_ms, created_at, updated_at)

query_runs(id, *workspace_id*†, *saved_query_id*†, sql_text, source, target, status, duration_ms, row_count, error_message, created_at)

nl_questions(id, *query_run_id*†, question, explanation, model)

explain_snapshots(id, *workspace_id*†, *query_run_id*†, target, plan, planning_ms, execution_ms, created_at)

index_experiments(id, *workspace_id*†, table_name, columns, index_name, ddl, before_ms, after_ms, runs, speedup, status, created_at)

Bookkeeping

schema_migrations(version, applied_at)

Figure 3.3: Relational schema of the `qlens` schema. Primary keys are underlined; foreign keys are in italics with † (targets in Table 3.1).

Deleting a parent removes its children, so deleting a user leaves no orphans; `SET NULL` is used only where history must survive.

3.3.4 Normalization of the Schema

Each table describes one kind of thing and every non-key attribute depends on the whole key and nothing else, so the schema is in BCNF apart from the deliberate cases below; in particular there are no transitive dependencies through non-key attributes. Lists are never stored in a column when they are facts in their own right: the columns of a candidate key, the left-hand side of a dependency and the columns of a unique key each live in a junction table, so "which dependencies use column *c*?" is an indexed join, not a string search. (`sample_values` and `index_experiments.columns` are arrays because they are display values, never queried by element.) Three denormalizations are deliberate. `dataset_id` is stored next to `column_id` in `duplicate_groups`, `dataset_fixes` and `functional_dependencies` although it follows from the column, so that the Analyze page filters by dataset without a join; `workspace_id` in `explain_snapshots` likewise saves a join through `query_runs` (and lets a plan without a run

Table 3.1: Relationships of the `qlens` schema (parent $1 : N$ child unless stated).

Parent	Child (foreign key)	Cardinality	ON DELETE
users	sessions (user_id), workspaces (owner_id)	$1 : N$	CASCADE
workspaces	datasets (workspace_id, unique)	$1 : 0..1$	CASCADE
workspaces	saved_queries, query_runs, explain_snapshots, index_experiments (workspace_id)	$1 : N$	CASCADE
datasets	dataset_columns, candidate_keys, functional_dependencies, duplicate_groups, dataset_fixes, schema_proposals (dataset_id)	$1 : N$	CASCADE
dataset_columns	functional_dependencies (rhs_column_id), duplicate_groups, dataset_fixes (column_id)	$1 : N$	CASCADE
candidate_keys × dataset_columns	candidate_key_columns (key_id, column_id)	$M : N$	CASCADE
functional_dependencies × dataset_columns	fd_lhs_columns (fd_id, column_id)	$M : N$	CASCADE
duplicate_groups	duplicate_values (group_id)	$1 : N$	CASCADE
schema_proposals	proposed_tables (proposal_id)	$1 : N$	CASCADE
proposed_tables	proposed_columns (table_id), proposed_unique_keys (table_id)	$1 : N$	CASCADE
proposed_tables	proposed_columns (references_table_id, nullable)	$0..1 : N$	CASCADE
dataset_columns	proposed_columns (source_column_id, nullable)	$0..1 : N$	SET NULL
proposed_unique_keys × proposed_columns	proposed_unique_key_columns (key_id, column_id)	$M : N$	CASCADE
saved_queries	query_runs (saved_query_id, nullable)	$0..1 : N$	SET NULL
query_runs	nl_questions (query_run_id, unique)	$1 : 0..1$	CASCADE
query_runs	explain_snapshots (query_run_id, nullable)	$0..1 : N$	CASCADE

belong to a workspace). Derived values such as `is_exact`, `violation_ratio` and `speedup` are `GENERATED ALWAYS AS (...) STORED`, so the redundancy can never become an anomaly: the database recomputes them on every write. `run_count` and `last_run_at` of a saved query are maintained by a trigger for the same reason.

3.3.5 Constraints and Indexes

Every column is `NOT NULL` unless absence has a meaning (a run without a saved query, an unchecked design). Closed vocabularies are `text` with `CHECK (x IN (...))`, and every stored name that will become an SQL identifier is checked against `^[a-z_][a-z0-9_]{0,62}$`. Table 3.2 lists the rules that carry business logic; Table 3.3 the secondary indexes, each justified by the query it serves (PostgreSQL does not index foreign keys by itself).

Table 3.2: Integrity rules that carry business logic.

Rule	Meaning
`CHECK (status <> 'applied' OR lossless_ok IS TRUE)`	no design is recorded as applied without a passed check; `IS TRUE` makes NULL fail
`UNIQUE (dataset_id) WHERE status = 'draft'` and `WHERE status = 'applied'`	at most one draft and one applied design per dataset (partial unique indexes)
`CHECK ((role = 'fk') = (references_table_id IS NOT NULL))`	a column is a foreign key exactly when it references a table
`CHECK ((status = 'ok') = (error_message IS NULL))`	a run has an error message exactly when it failed
`CHECK (email = lower(email) AND position('@' in email) > 1)`	makes `UNIQUE(email)` case-insensitive
`CHECK (octet_length(token_hash) = 32)`, `CHECK (expires_at > created_at)`	sessions store only a SHA-256 digest and cannot expire before they start
`CHECK (violating_rows <= row_count)`, `CHECK (key_size BETWEEN 1 AND 3)`	analysis measures stay consistent with the search limits

Table 3.3: Secondary indexes of the `qlens` schema and the queries they serve.

Index	Serves
`query_runs (workspace_id, created_at DESC)`	history page, `v_workspace_summary`, `v_slow_queries`
`query_runs (saved_query_id, created_at) WHERE saved_query_id IS NOT NULL`	`v_saved_query_trend`; partial, as most runs are ad hoc
`dataset_fixes (dataset_id, created_at DESC)`	edit history of a dataset, newest first
`candidate_keys (dataset_id, key_size)`	keys of a dataset, smallest first
`functional_dependencies (dataset_id), (rhs_column_id)`	dependencies of a dataset; of a column
`explain_snapshots`, `index_experiments (workspace_id, created_at DESC)`	latest plans and experiments of a workspace
`proposed_columns (references_table_id) WHERE ... IS NOT NULL`	relationship lines of the design diagram
`sessions (user_id)`; `*_columns (column_id)`	cascades from `users` and `dataset_columns`

3.3.6 Triggers, Views, Functions and Roles

Triggers. `touch_updated_at` is a `BEFORE UPDATE` row trigger on workspaces, datasets and saved queries; it uses `clock_timestamp()` because `now()` is frozen for the whole transaction. `record_query_run` (Listing 3.1) keeps a saved query's counters in step with its successful runs; the `WHEN` clause filters before the function is even called.

Listing 3.1: AFTER INSERT trigger on `query_runs` (`002_logic.sql`)

```
CREATE FUNCTION qlens.record_query_run() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN
    UPDATE qlens.saved_queries
       SET run_count = run_count + 1, last_run_at = NEW.created_at,
           last_duration_ms = NEW.duration_ms
     WHERE id = NEW.saved_query_id;
    RETURN NULL;   -- AFTER trigger: return value is ignored
```

```
END $$;
CREATE TRIGGER query_runs_record AFTER INSERT ON qlens.query_runs
    FOR EACH ROW WHEN (NEW.saved_query_id IS NOT NULL AND NEW.status = 'ok')
    EXECUTE FUNCTION qlens.record_query_run();
```

Views. `v_workspace_summary` gives the dashboard one row per workspace using two `LEFT JOIN LATERAL` aggregates (an aggregate without `GROUP BY` always returns a row, so empty workspaces are kept) and `GREATEST` over three timestamps. `v_slow_queries` keeps the runs at or above their workspace's 90th percentile, computed with `percentile_cont(0.9) WITHIN GROUP (ORDER BY duration_ms)`. `v_saved_query_trend` numbers each saved query's runs and adds the previous duration and a five-run moving average with the window functions `row_number()`, `lag()` and `avg() OVER (... ROWS BETWEEN 4 PRECEDING AND CURRENT ROW)`.

PL/pgSQL functions. `try_date(text)` parses ISO and day-first dates and returns NULL instead of raising, so type inference can test every value; `check_fd` (Listing 3.3) and `is_unique` measure dependencies and keys with dynamic SQL in which identifiers pass only through `format('%I')`.

Roles and privileges. Creating a workspace runs, in the same transaction, `CREATE ROLE qlens_ws_<id> LOGIN PASSWORD ... NOINHERIT CONNECTION LIMIT 3`, then `ALTER ROLE ... SET default_transaction_read_only = on, statement_timeout = '5s', idle_in_transaction_session_timeout = '10s'` and the `search_path`, and for each of its schemas `GRANT USAGE ON SCHEMA`, `GRANT SELECT ON ALL TABLES IN SCHEMA` and `ALTER DEFAULT PRIVILEGES ... GRANT SELECT ON TABLES`. The role gets nothing on `qlens` or on any other workspace. Its password is an HMAC of the workspace id under the server secret, so it is never stored, and deleting the workspace runs `DROP OWNED BY` and `DROP ROLE`.

3.3.7 SQL Used by the Application

Every feature is a piece of hand-written SQL (Table 3.4); the application code mostly composes identifiers safely (`psycopg.sql.Identifier`) and binds values.

Listing 3.2 is the core of the near-duplicate search (`cleanup.py`). The `MATERIALIZED` CTE forces the index-backed `%` join to run first; the outer filters then drop pairs whose digits differ and pairs of equally frequent values.

Listing 3.2: Near-duplicate search with `pg_trgm` over a GIN index (from `cleanup.py`)

```
CREATE INDEX ON pg_temp._qlens_vals USING gin (val gin_trgm_ops);
SELECT set_config('pg_trgm.similarity_threshold', '0.5', true);
WITH m AS MATERIALIZED (SELECT a.val AS a, a.n AS an, b.val AS b, b.n AS bn
                        FROM pg_temp._qlens_vals a JOIN pg_temp._qlens_vals b
                        ON a.val % b.val AND a.val < b.val)
SELECT a, an, b, bn, similarity(a, b) AS sim FROM m
WHERE regexp_replace(a, '[^0-9]', '', 'g') = regexp_replace(b, '[^0-9]', '', 'g')
  AND least(an, bn) <= greatest(an, bn) * 0.333;
```

Table 3.4: SQL used by each feature of QLens.

Feature	SQL it runs
Import	`COPY _raw FROM STDIN`; type tests `bool_and(col ~ regex)`; `CREATE TABLE _staging AS SELECT` with casts
Profiling	one pass of `count(col)`, `count(DISTINCT col)`; top values with `GROUP BY ... ORDER BY count(*) DESC LIMIT 5`
Dependencies, keys	`qlens.check_fd` (two-level `GROUP BY`, `FILTER`); `qlens.is_unique` (`count(*)` vs. `SELECT DISTINCT`)
Near-duplicates	temp table with GIN `gin_trgm_ops` index; self-join on `%`; `similarity()`; window `count(*) OVER (PARTITION BY norm)`
Inconsistencies, fixes	window `count(*) OVER (PARTITION BY lhs)`; `UPDATE ... FROM` a `FOR UPDATE` CTE where `IS DISTINCT FROM`; audit `INSERT`
Design	generated `CREATE TABLE`, `INSERT ... SELECT DISTINCT`, natural-key `LEFT JOIN` lookups, `CREATE VIEW`; `quote_ident` for display
Apply	one transaction, `SELECT ... FOR UPDATE`, `EXCEPT ALL` both ways, `ANALYZE`; schema read back from `pg_class`, `pg_attribute`, `pg_constraint`, `pg_index`
Query runner	new login as the workspace role, `BEGIN READ ONLY`, `set_config('statement_timeout', ..., true)`, `DECLARE ... CURSOR FOR`, `FETCH FORWARD 1001`, `ROLLBACK`
History, dashboard	`INSERT INTO query_runs` (trigger), `v_saved_query_trend`, `v_slow_queries`, `v_workspace_summary`
Accounts	`INSERT ... ON CONFLICT DO NOTHING RETURNING id`; session lookup by token digest with `expires_at > now()`

3.3.8 Dependency Discovery and Cleaning

Figure 3.4 shows the pipeline from upload to apply. `qlens.check_fd` (Listing 3.3) returns, for $X \to A$, the rows that would have to change for the dependency to hold: per X-group, every row except those with the most frequent A, the g_3 measure of approximate dependencies [10]. A dependency is exact at 0, approximate up to 2% of rows, and needs at least two repeated X-groups. Keys are tested with `is_unique` on 1–3 columns after pruning by the profile (no NULLs; the product of distinct counts must reach the row count). Measures are never determinants, a pair is dropped when one of its columns already determines the target, an approximate dependency explained through an exact one is reported once at its root, and implied dependencies are flagged *derived*. On the fee register six core dependencies remain, two approximate (8 and 5 rows).

Listing 3.3: `qlens.check_fd`: measuring a dependency with dynamic SQL (`004_analysis.sql`)

```
CREATE FUNCTION qlens.check_fd(p_schema text, p_table text, p_lhs text[], p_rhs text)
RETURNS TABLE (row_count bigint, lhs_values bigint, repeated_groups bigint, violating_rows
    bigint)
LANGUAGE plpgsql STABLE AS $$
DECLARE
    t   text := format('%I.%I', p_schema, p_table);
    lhs text;
BEGIN
    SELECT string_agg(format('%I', c), ', ') INTO lhs FROM unnest(p_lhs) AS c;
    RETURN QUERY EXECUTE format(
        'WITH c AS (SELECT %1$s, %2$I, count(*) AS cnt FROM %3$s GROUP BY %1$s, %2$I), '
```

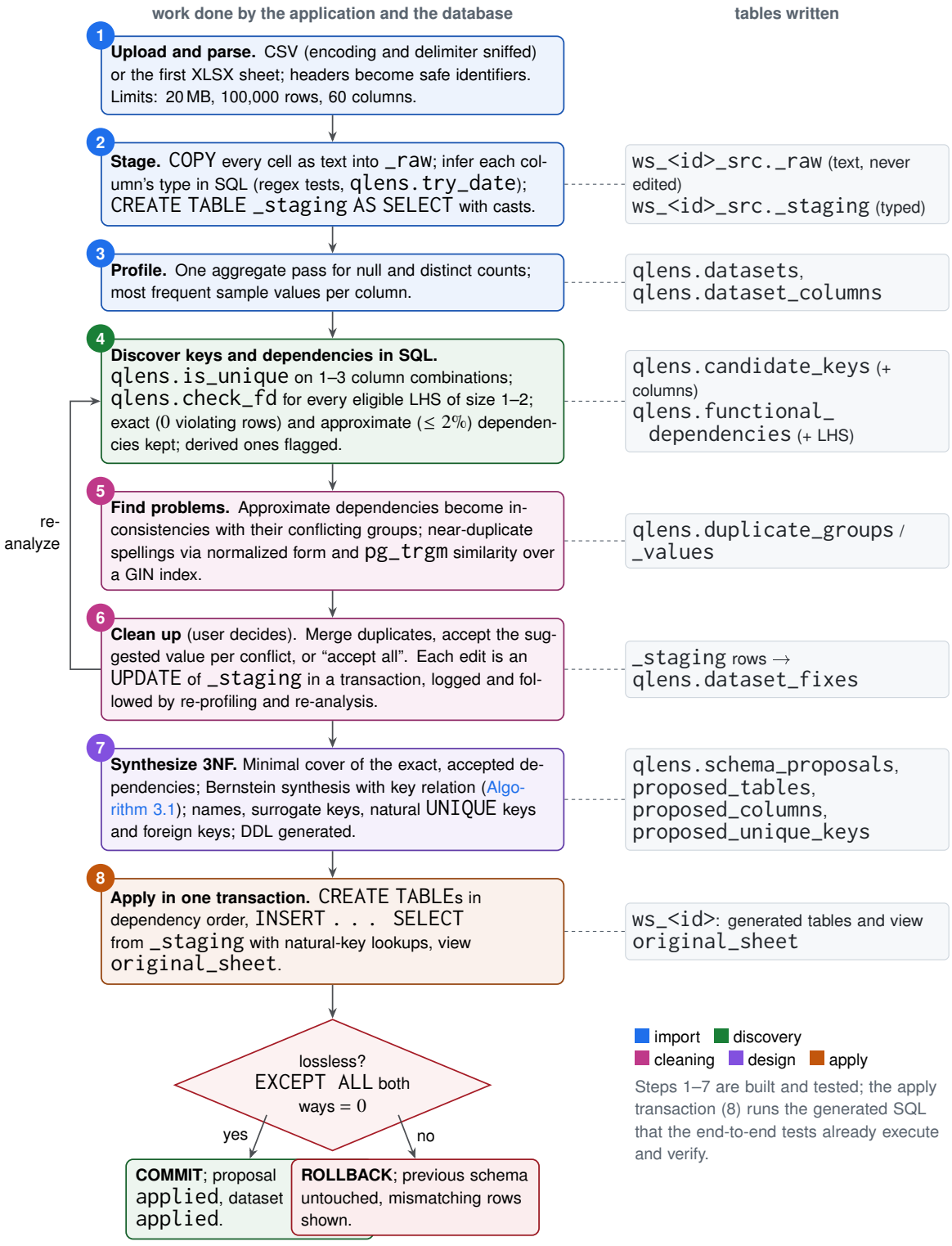


Figure 3.4: The normalization pipeline from upload to the lossless check. Left: the steps in order (colour marks the phase); right: the tables each step writes. Steps 4–6 can loop: every clean-up re-profiles the edited columns and re-runs discovery.

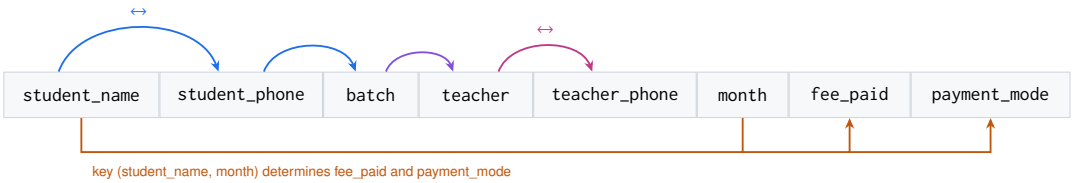


Figure 3.5: Dependency diagram of the fee register: arrows point from determinant to dependent, ↔ marks mutually determining columns, and colours match the tables of Figure 3.6. Before cleaning, {student_phone} → student_name fails on 8 rows and {teacher} → teacher_phone on 5.

```
        '     g AS (SELECT %1$s, sum(cnt) AS n, max(cnt) AS keep FROM c GROUP BY %1$s) '
        'SELECT (SELECT count(*) FROM %3$s), count(*), count(*) FILTER (WHERE n >= 2), '
        '       coalesce(sum(n - keep), 0)::bigint FROM g',
        lhs, p_rhs, t);
END;
$$;
```

Cleaning edits only `_staging`. Spellings equal after lower-casing and collapsing spaces are certain duplicates; probable typos come from Listing 3.2 [11], clustered with union–find. Each approximate dependency lists its conflicting groups with the most frequent value as the suggestion (the minimal repair under g_3). Every merge or fix is one transactional `UPDATE`, logged in `dataset_fixes` and followed by re-profiling; "accept all" does all of them and re-analyzes.

3.3.9 3NF Synthesis

The design is built from exactly the dependencies the user sees: exact, accepted and not derived. Closure, minimal cover and synthesis (Algorithm 3.1) are set algebra in Python: $X \to A$ follows from F iff $A \in X^+$ [12]; groups with equivalent determinants are merged and Bernstein's step 5 prunes what the equivalences make redundant [3]. Synthesis alone is lossless only if some relation contains a key of the whole table [13], so QLens designates as *fact* the relation that contains every undetermined attribute and determines everything, or adds one built from a key of the dependencies. Joining outward from the fact, each join is on a key of the relation joined, so rows are neither added nor lost; 900 randomized cases are checked against a chase-based lossless test [14] and a brute-force 3NF test.

3.3.10 Generated Database and Apply

Built Each synthesized relation becomes a table with `id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY`. Its determinant and equivalent determinants become `UNIQUE NULLS NOT DISTINCT` constraints, so an empty *Teacher* cell is one teacher stored once. A relation that contains another entity's natural key stores a foreign key `<entity>_id ... ON DELETE RESTRICT` instead, `NOT NULL` when the replaced columns had no empty cells. Tables are named from their determinant, explained in plain English and renamable while the design is a draft (409 `proposal_not_draft` afterwards). Figure 3.6 is the result for the fee register.

The script is generated in foreign-key order: entity loads are `INSERT ... SELECT DISTINCT` from `_staging`; foreign keys are looked up by natural key with `=` where the key has no NULLs

Algorithm 3.1. Lossless 3NF synthesis used by QLens (Bernstein synthesis with equivalence merging, the step-5 redundancy pruning and a key relation)

Input: attributes U in sheet order; exact, accepted dependencies F; candidate keys $K_1, \ldots, K_m$ found in the data

Output: relations $R_1, \ldots, R_n$ (entities) and one fact relation R_f; every R_i is in 3NF and $\bowtie_i R_i$ is lossless

$G \leftarrow \textsc{MinimalCover}(F)$ `// singleton RHS, left-reduced, no redundant FD`
partition G into groups $G_X = \{X \to A \in G\}$, one per left-hand side X
foreach *pair of groups* G_X, G_Y **do**
  if $X \subseteq Y_G^+$ ***and*** $Y \subseteq X_G^+$ **then** `// equivalent determinants`
    merge G_X and G_Y into one group with determinants $\{X, Y\}$
$J \leftarrow \{X \to B \mid X, Y$ determinants of one merged group, $B \in Y\}$
for $X \to A \in G$ ***from last to first*** **do** `// Bernstein step 5`
  if $A \in X^+_{(G \setminus \{X \to A\}) \cup J}$ **then** remove $X \to A$ from G
foreach *merged group with determinants* $X_1, \ldots, X_k$ **do**
  $R \leftarrow X_1 \cup \cdots \cup X_k \cup \{A \mid X_j \to A \in G\}$; key $\leftarrow$ earliest X_j in sheet order
  emit entity relation R unless it carries no dependency
$D \leftarrow$ attributes appearing on the right of some emitted relation's non-key part
if *some emitted* R *contains* $U \setminus D$ ***and*** $R_G^+ = U$ **then**
  mark R as the fact relation R_f
else
  $Z \leftarrow U$
  for $a \in U$ ***in reverse sheet order*** **do** `// FD-key by reverse reduction`
    if $(Z \setminus \{a\})_G^+ = U$ **then** $Z \leftarrow Z \setminus \{a\}$
  $R_f \leftarrow Z$; its unique key is K_1 if $K_1 \subseteq Z$, else none (surrogate key only)
drop every entity relation whose attributes are a subset of another relation's
return *entity relations, then* R_f

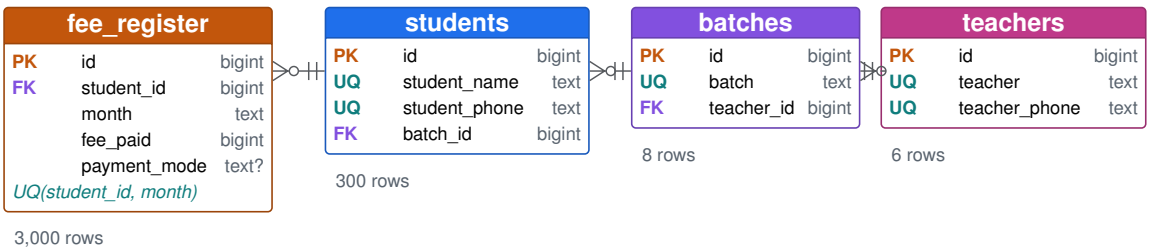


Figure 3.6: ER diagram of the database QLens generates for the fee register. Every table has a surrogate identity key; natural keys are `UNIQUE NULLS NOT DISTINCT`; foreign keys are `NOT NULL` and `ON DELETE RESTRICT`; `?` marks the only nullable column. Row counts are those of the applied design.

(so a hash join is possible) and `IS NOT DISTINCT FROM` otherwise; the fact keeps every row; and the view `original_sheet` joins everything back. Apply runs it in one transaction: lock the draft (`FOR UPDATE`), drop and recreate the workspace schema, execute, and run Listing 3.4. A non-zero count raises 409 `lossless_check_failed` with up to five differing rows and PostgreSQL rolls everything back. Otherwise the old applied design becomes *superseded* before the new one becomes *applied* (the partial unique indexes allow only one), and the new tables are `ANALYZE`d.

Listing 3.4: Lossless check generated for the fee register (`cols` abbreviates the eight sheet columns)

```
SELECT (SELECT count(*) FROM ws_20_src._staging)  AS staging_rows,
       (SELECT count(*) FROM ws_20.original_sheet) AS reconstructed_rows,
       (SELECT count(*) FROM (SELECT cols FROM ws_20.original_sheet
                              EXCEPT ALL SELECT cols FROM ws_20_src._staging) d) AS
    extra_rows,
       (SELECT count(*) FROM (SELECT cols FROM ws_20_src._staging
                              EXCEPT ALL SELECT cols FROM ws_20.original_sheet) d) AS
    missing_rows;
```

`EXCEPT ALL` compares multisets, so a lost or duplicated copy of a repeated row is counted, and both directions are needed because A `EXCEPT ALL` B is empty whenever $A \subseteq B$. On the cleaned fee register all 3,000 rows come back exactly; tests also cover forced failures, reserved-word headers, empty determinants, the other samples and random sheets (a 400-seed run passed throughout).

Example queries. Listing 3.5 shows queries a user runs on the generated database in the Query screen; on the applied fee register they return 19 students unpaid for October, 6 batches with more than 25 unpaid months (JEE Evening, Mr. Gupta, first with Rs. 12,75,800 collected) and one ranked row for each of the 300 students.

Listing 3.5: Example queries on the generated fee-register database (runnable in the Query screen)

```
-- 1. Who has not paid for October? (joins)
SELECT s.student_name, s.student_phone, b.batch
FROM fee_register f JOIN students s ON s.id = f.student_id
                    JOIN batches  b ON b.id = s.batch_id
WHERE f.month = 'Oct' AND f.fee_paid = 0 ORDER BY b.batch, s.student_name;
-- 2. Collection per batch and teacher, only batches with many unpaid months
SELECT b.batch, t.teacher, sum(f.fee_paid) AS collected,
       count(*) FILTER (WHERE f.fee_paid = 0) AS unpaid
FROM fee_register f JOIN students s ON s.id = f.student_id
     JOIN batches b ON b.id = s.batch_id JOIN teachers t ON t.id = b.teacher_id
GROUP BY b.batch, t.teacher
HAVING count(*) FILTER (WHERE f.fee_paid = 0) > 25 ORDER BY collected DESC;
-- 3. Rank students within their batch by total paid (window function)
SELECT batch, student_name, total,
       rank() OVER (PARTITION BY batch ORDER BY total DESC) AS rank_in_batch
FROM (SELECT b.batch, s.student_name, sum(f.fee_paid) AS total
      FROM fee_register f JOIN students s ON s.id = f.student_id
           JOIN batches b ON b.id = s.batch_id
      GROUP BY b.batch, s.student_name) t
ORDER BY batch, rank_in_batch;
```

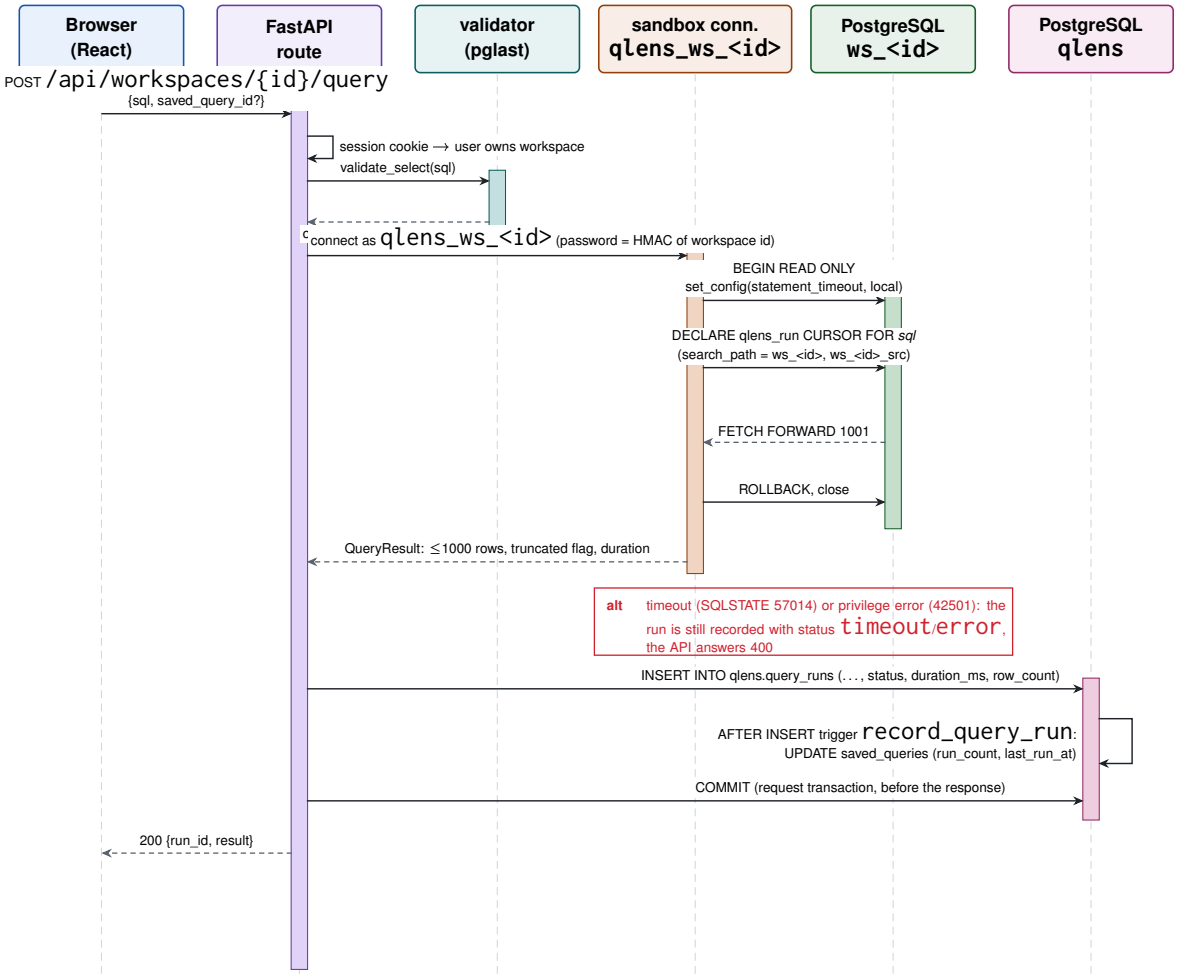


Figure 3.7: Sequence of a user query through the sandbox. The query never runs on the application's pooled connection: it runs on a separate login as the workspace role, whose privileges reach only that workspace's schemas, inside a read-only transaction with a statement timeout and a row cap. The run is then recorded on the pooled connection, where a trigger maintains the saved query's statistics.

3.3.11 Query Sandbox

Built A user query never runs on the application's pooled connection (Figure 3.7). The validator parses it with pglast, the PostgreSQL parser itself, walking the tree iteratively with a token cap: it must be exactly one `SELECT` (set operations allowed), without `INTO`, locking clauses or data-modifying CTEs, and without functions on a denylist (`set_config`, `pg_read_file`, `dblink`, `query_to_xml` and others). The validator gives friendly errors but is not the security boundary. The runner opens a fresh login as `qlens_ws_<id>`, starts a `READ ONLY` transaction, sets a local `statement_timeout` of at most 5 s, declares a cursor for the query with the extended protocol (which refuses a second statement) and fetches 1,001 rows, so that a 1,000-row cap and a *truncated* flag need no full materialization; it always rolls back. A timeout is reported as SQLSTATE 57014. Every run, failed or not, is then inserted into `query_runs`, where the trigger of Listing 3.1 updates the saved query.

3.3.12 Interface and Assistant

Each generated table keeps one lane colour (`proposed_tables.color_index`) on the Analyze diagram, the design canvas, the SQL and these figures. The landing page, dashboard, Analyze screen (keys banner and a dependency graph, dashed when approximate) and the design canvas are Built. The canvas (Figure 3.8) shows the proposal as draggable tables on a dot grid with inline renaming, the reasons for each table, the SQL and the Apply button with the lossless result. Motion is staged and uses only `transform`, `opacity` and `stroke-dashoffset`; it stops under `prefers-reduced-motion`.

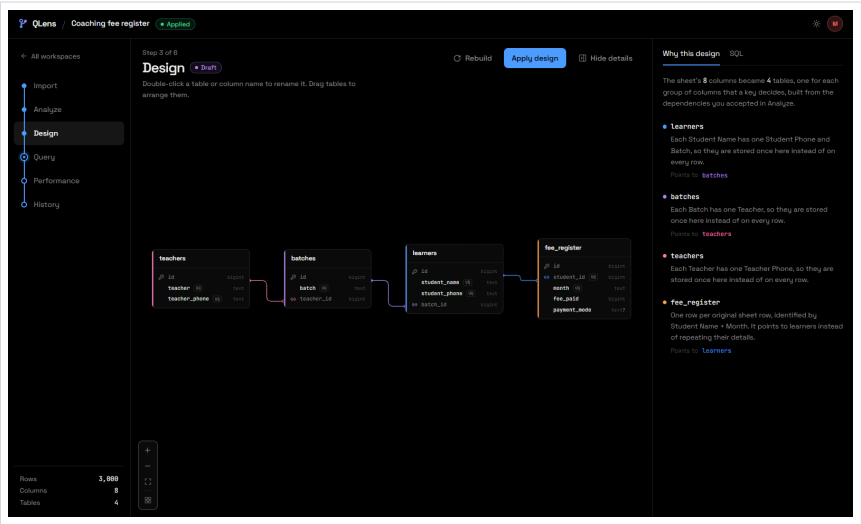


Figure 3.8: The design canvas: the proposed fee-register schema as an ER diagram, with the reasons for each table and the Apply action.

In progress An optional Gemini assistant floats over the canvas as a command bar. By function calling it either answers a question or reworks the draft, but only through server-validated tools: `describe_schema`, `check_dependency` (runs `qlens.check_fd`), `run_sql` (through the sandbox), `explain_query`, `rename_table`/`rename_column`, `set_dependency_accepted` (re-synthesizes) and `move_column`, allowed only when `check_fd` shows the move is exact. Edits create new draft versions, so they can be undone, and conversations are stored in `qlens`.

CHAPTER 4

Conclusion

QLens shows that a person who owns only a messy spreadsheet can be taken, step by step and with every step explained, to a verified third-normal-form database. Type inference, key search and dependency discovery run inside PostgreSQL through PL/pgSQL and dynamic SQL; near-duplicates are found with `pg_trgm` over a GIN index; repairs are transactional and audited; the synthesis is Bernstein's algorithm with a key relation, property-tested for 3NF and losslessness; and the apply step proves on the real rows, with `EXCEPT ALL` in both directions inside one transaction, that nothing was lost. On the fee register every planted problem is found and the expected four-table design rebuilds all 3,000 rows exactly. Around this sits a 22-table metadata schema that uses generated columns, `CHECK` rules, partial unique indexes, triggers and window-function views, and a per-workspace login role that isolates users by privilege. 302 backend and 63 frontend tests pass.

Lessons. Dependencies in real data are statistical before they are logical, so measuring violations (g_3) and letting the user decide was the central design decision. Normalization needs clean data first: a rule broken by eight typos would otherwise be lost and the design would keep a redundant column. Set-oriented checks belong in the database. Constraints are documentation that executes: they caught our own mistakes, such as an applied-design rule that let `NULL` pass until it was written as `IS TRUE`. And losslessness, a theorem in the textbook, becomes in practice a query.

Limitations. Left-hand sides are limited to two columns and keys to three; noise filtering relies on thresholds, so the user keeps the final say; one flat sheet of up to 100,000 rows is normalized at a time, and only to 3NF; trigram similarity cannot know that two people share a name; and the evidence comes from synthetic samples and tests, not a user study. The assistant is In progress, and the performance laboratory and step-through visualizer are Planned.

Future work. Completing the validated assistant; a performance laboratory that renders `EXPLAIN (ANALYZE, BUFFERS)` as a tree and times index experiments on a scaled copy; a clause-by-clause visualizer for teaching; wider dependency search with sampling and multivalued dependencies for 4NF; and resource limits for hosted, multi-tenant use, which the per-workspace role design was chosen to make possible.

References

[1] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[2] W. Kent, "A simple guide to five normal forms in relational database theory," *Communications of the ACM*, vol. 26, no. 2, pp. 120–125, 1983.

[3] P. A. Bernstein, "Synthesizing third normal form relations from functional dependencies," *ACM Transactions on Database Systems*, vol. 1, no. 4, pp. 277–298, 1976.

[4] The PostgreSQL Global Development Group, "PostgreSQL 18 documentation," https://www.postgresql.org/docs/18/, 2025.

[5] K. Beck, *Test-Driven Development: By Example*. Addison-Wesley, 2002.

[6] Meta Open Source, "React documentation," https://react.dev/, 2025.

[7] S. Ramírez, "FastAPI documentation," https://fastapi.tiangolo.com/, 2025.

[8] D. Varrazzo, "Psycopg 3: PostgreSQL database adapter for Python," https://www.psycopg.org/psycopg3/docs/, 2025.

[9] L. Gaifax, "pglast: PostgreSQL languages AST and statements prettifier," https://pglast.readthedocs.io/, 2025.

[10] J. Kivinen and H. Mannila, "Approximate inference of functional dependencies from relations," *Theoretical Computer Science*, vol. 149, no. 1, pp. 129–149, 1995.

[11] R. C. Angell, G. E. Freund, and P. Willett, "Automatic spelling correction using a trigram similarity measure," *Information Processing & Management*, vol. 19, no. 4, pp. 255–261, 1983.

[12] W. W. Armstrong, "Dependency structures of data base relationships," in *Proceedings of the IFIP Congress*, 1974, pp. 580–583.

[13] J. Biskup, U. Dayal, and P. A. Bernstein, "Synthesizing independent database schemas," in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1979, pp. 143–151.

[14] A. V. Aho, C. Beeri, and J. D. Ullman, "The theory of joins in relational databases," *ACM Transactions on Database Systems*, vol. 4, no. 3, pp. 297–314, 1979.

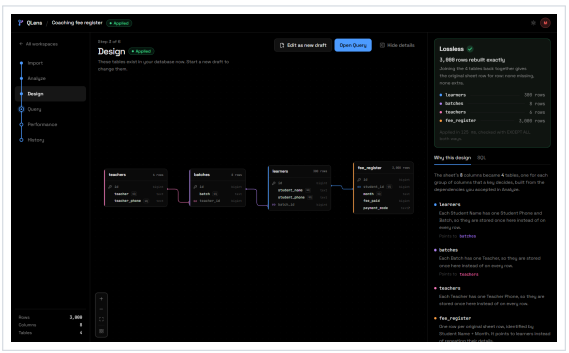
CHAPTER A

Sample Datasets and Screens

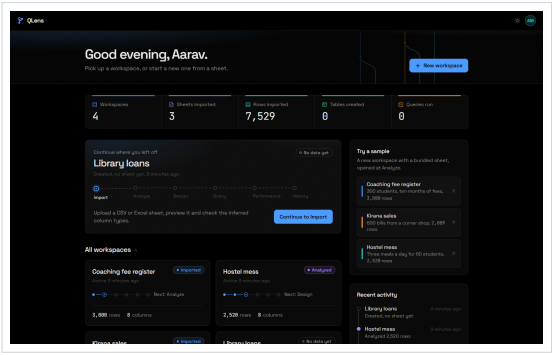
The three bundled sheets are generated deterministically (seed 2026), and their planted problems are asserted by the tests, so every demonstration sees identical numbers.

Table A.1: Bundled sample sheets and their planted problems.

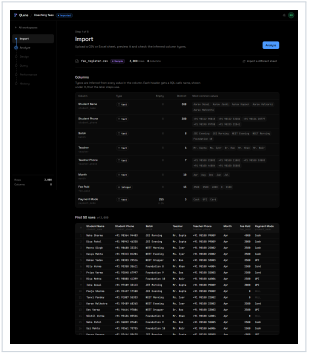
Sheet	Rows	Structure	Planted problems
Coaching fee register	3,000	300 students × 10 months; 8 batches, 6 teachers; key (student, month); about 8% unpaid and 15% partial payments	4 names in variant spellings (8 rows); Mr. Gupta's old phone (5 rows); a genuine look-alike, Priya Sharma, must not be merged
Kirana sales	2,009	600 bills, 80 customers, 40 items in 6 categories; key (bill, item); amount = price × qty	old price of Toor Dal (3 rows); *Mohan lal* for *Mohan Lal* (3 rows)
Hostel mess	2,520	60 students × 14 days × 3 meals; room → block → warden; one menu item per (date, meal)	*Mrs Rao* for *Mrs. Rao* (2 rows)



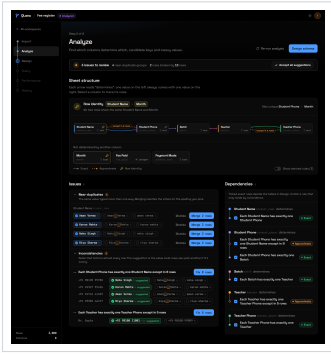
(a) Design applied: lossless result



(b) Workspace dashboard



(c) Import: inferred types



(d) Analyze: keys, dependencies, issues

Figure A.1: Further screens of QLens.